

Desarrollo de módulos para Odoo

CASO PRÁCTICO. MÓDULOS, VISTAS, ACCIONES



Licencia



Reconocimiento – NoComercial - CompartirIgual (BY-NC-SA): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

 **Importante**

 **Atención**

 **Interesante**

1. Introducción	3
2. ¿Qué vamos a aprender?	3
3. ¿Qué ocurre cuando lo instalamos?	3
4. Análisis del archivo <code>__manifest__.py</code>	4
5. Archivos de inicialización (<code>__init__.py</code>)	5
6. ¿Qué es un modelo en Odoo?	5
7. ¿Qué es <code>views.xml</code> ?	7
8. Seguridad en Odoo: <code>ir.model.access.csv</code>	11
9. Resumen final del módulo	12

UNIDAD 04. CASO PRÁCTICO 01

1. INTRODUCCIÓN

En esta guía vamos a estudiar en profundidad cómo está construido un módulo de Odoo que ya está completamente desarrollado. No vamos a programarlo paso a paso, sino que vamos a **desmenuzarlo, entenderlo, y aprender por qué está hecho así**.

Este módulo se llama **“Lista de Tareas”**. Es un ejemplo simple, pero completo, pensado para estudiantes que están dando sus primeros pasos en el desarrollo con Odoo. A través de él, vas a aprender los pilares básicos del desarrollo backend y frontend en este framework.

Porque contiene, en forma compacta, todo lo que necesitas para empezar a comprender cómo funciona el desarrollo de aplicaciones en Odoo:

- Un **modelo de datos** con lógica calculada.
- Una **interfaz completa** con formularios, tablas y menús.
- Un sistema de **permisos de acceso** para los usuarios.
- Un módulo funcional que se puede instalar y usar desde la interfaz web.

Además, está escrito con claridad, buenas prácticas y con muchos puntos donde podrías ampliarlo o mejorarlo.

2. ¿QUÉ VAMOS A APRENDER?

Leyendo esta guía y entendiendo este módulo vas a aprender a:

- Interpretar un archivo `models.py` y entender cómo se definen los campos.
- Comprender cómo Odoo transforma esos datos en una interfaz gráfica.
- Leer una vista XML y saber qué hace cada parte.
- Aplicar correctamente permisos para que el módulo sea seguro y accesible.
- Conectar cada archivo con su rol dentro del sistema.

3. ¿QUÉ OCURRE CUANDO LO INSTALAMOS?

En cuanto el módulo se instala:

1. **Aparece un nuevo menú** en Odoo llamado *“Listado de tareas”*.
2. El usuario puede entrar y ver una tabla vacía (lista de tareas).
3. Puede crear tareas desde un formulario.
4. Al poner una prioridad, el sistema calcula automáticamente si es urgente.
5. Puede marcar la tarea como *“realizada”*.

Todo eso se consigue con muy poco código, pero bien estructurado. Y ese código está repartido en **varios archivos** que trabajan juntos.

El módulo tiene cinco archivos principales:

Archivo	¿Qué hace?
__manifest__.py	Informa a Odoo sobre el módulo y qué debe cargar
__init__.py	Indica a Odoo qué partes de Python usar
models.py	Define cómo son las tareas (modelo de datos)
views.xml	Define cómo se ven las tareas en la pantalla
ir.model.access.csv	Indica quién puede ver y usar las tareas

Cada uno tiene una función específica. No están hechos al azar.

Son como piezas de un rompecabezas que, juntas, crean una aplicación dentro de Odoo.

4. ANÁLISIS DEL ARCHIVO __MANIFEST__.PY

Este archivo es esencial. Odoo lo usa para identificar y cargar correctamente el módulo.

```
{
    'name': "Lista de tareas",
    'summary': "Módulo educativo para crear una lista de tareas simples",
    'description': """Este módulo permite crear tareas con prioridad, urgencia
calculada y estado de realización. Ideal como primer proyecto de desarrollo en
Odoo."""",
    'author': "Tu Nombre",
    'website': "https://tusitio.com",
    'application': True,
    'category': 'Productivity',
    'version': '0.1',
    'license': 'LGPL-3',
    'installable': True,
    'depends': ['base'],
    'data': [
        'security/ir.model.access.csv',
        'views/views.xml',
    ]
}
```

Campos importantes:

- **name:** Nombre visible en la lista de módulos.
- **summary y description:** Breve y detallada descripción del módulo.
- **depends:** Lista de módulos requeridos. Aquí solo se requiere 'base', suficiente para módulos básicos.

- **data:** Lista de archivos XML y CSV que se cargarán al instalar el módulo (vistas, seguridad, datos predefinidos).
- **application:** Si es True, aparecerá como una aplicación en el dashboard principal.

5. ARCHIVOS DE INICIALIZACIÓN (`__init__.py`)

En este archivo indicamos a Python qué archivos deben cargarse cuando se importa el módulo.

```
from . import models
```

Esto le dice a Odoo que cargue el archivo `models.py`, donde se encuentra la lógica de negocio del módulo.

6. ¿QUÉ ES UN MODELO EN ODOO?

Un **modelo** en Odoo es como una *plantilla* para guardar información en la base de datos. Por ejemplo, si queremos guardar tareas, necesitamos un modelo que tenga campos como el nombre de la tarea, si está realizada, qué tan prioritaria es, etc.

En términos técnicos, un modelo es una clase de Python que hereda de `models.Model`.

Contenido del archivo `models.py`

Aquí tienes el código completo del archivo con explicaciones paso a paso.

```
from odoo import models, fields, api
```

♦ ¿Qué hace esta línea?

Importa las clases necesarias de Odoo:

- `models`: Para crear nuevos modelos de datos.
- `fields`: Para definir los tipos de campos (texto, número, booleano, etc.).
- `api`: Para usar decoradores como `@api.depends`, que indican que un campo se calcula automáticamente.

Definición del modelo

```
class ListaTareas(models.Model):
```

- Se está creando una clase llamada `ListaTareas`, que es nuestro modelo.
- Hereda de `models.Model`, lo que significa que sigue las reglas de Odoo para guardar datos.

Identificadores internos del modelo

```
_name = 'lista_tareas.lista'  
_description = 'Modelo de la lista de tareas'  
_rec_name = "tarea"
```

Atributo	Descripción
_name	Nombre técnico del modelo. Importante: debe ser único. Se usa en vistas, menús, seguridad.
_description	Texto visible en herramientas administrativas o documentación interna.
_rec_name	Campo que se mostrará como “nombre” del registro en listas y relaciones.

En este caso, cuando se vea una lista de registros de este modelo, lo que se mostrará es el contenido del campo tarea.



Campos definidos en el modelo

```
tarea = fields.Char(string="Tarea")
```

- **Tipo:** Char (texto corto)
- **Finalidad:** Almacena el nombre de la tarea.
- **string="Tarea":** Nombre visible en la interfaz de usuario (en vez de mostrar tarea, se mostrará “Tarea”).

```
prioridad = fields.Integer(string="Prioridad")
```

- **Tipo:** Integer (número entero)
- **Uso:** Representa la importancia de la tarea. Por ejemplo, 5 es poco importante, 90 es muy importante.

```
urgente = fields.Boolean(string="Urgente", compute="_value_urgente", store=True)
```

- **Tipo:** Boolean (Sí/No o Verdadero/Falso)
- **Significado:** Se activa automáticamente si la prioridad es alta.
- **compute:** Significa que no lo llena el usuario, sino que se calcula automáticamente usando una función.
- **store=True:** Se guarda el resultado en la base de datos, lo que permite buscar y ordenar por este campo.

```
realizada = fields.Boolean(string="Realizada")
```

- **Tipo:** Boolean
- **Función:** Permite marcar si la tarea ya fue completada.



Campo calculado: urgente

Este campo se calcula automáticamente. Aquí está la función que realiza ese cálculo:

```
@api.depends('prioridad')
def _value_urgente(self):
    for record in self:
        record.urgente = record.prioridad > 10
```

Explicación paso a paso:

- `@api.depends('prioridad')`: Esta línea indica que si cambia el campo prioridad, se debe volver a ejecutar la función `_value_urgente`.
- `for record in self::`: En Odoo, las funciones de modelo siempre trabajan con *varios registros al mismo tiempo*, por eso usamos un `for`.
- `record.urgente = record.prioridad > 10`: Aquí ocurre la magia. Si la prioridad es mayor que 10, se marca como urgente (True), si no, queda como no urgente (False).

 Resumen de lo aprendido

Campo	Tipo	Editable por el usuario	Calculado automáticamente	Comentario
tarea	Char	✓	✗	Nombre de la tarea
prioridad	Integer	✓	✗	Valor numérico, de 1 a 100 por ejemplo
urgente	Boolean	✗	✓	Se activa si prioridad > 10
realizada	Boolean	✓	✗	Permite marcar una tarea como completada

7. ¿QUÉ ES VIEWS.XML?

El archivo `views.xml` define **cómo se visualiza y navega** el modelo en la interfaz de Odoo. Aunque en el archivo `models.py` definimos la estructura de los datos, en `views.xml` se especifica:

1. Qué menús tendrá el módulo.
2. Qué acciones se ejecutan al hacer clic en esos menús.
3. Cómo se ve la lista de tareas (vista tipo **lista**).
4. Cómo se ve cada tarea individual (vista tipo **formulario**).
5. Cómo buscar y filtrar tareas (vista tipo **búsqueda**).

Todo esto se escribe en **XML**, un lenguaje de etiquetas similar a HTML.

 Estructura general de views.xml

```
<odoo>
  <data>
    <!-- Aquí van todas las definiciones de vistas, menús y acciones -->
  </data>
</odoo>
```

- `<odoo>`: Raíz del archivo. Indica que este XML está destinado a Odoo.
- `<data>`: Contenedor principal de todos los elementos que vamos a declarar.

1. Acción principal (act_window)

```
<record model="ir.actions.act_window" id="action_lista_tareas">
  <field name="name">Listado de tareas</field>
  <field name="res_model">lista_tareas.lista</field>
  <field name="view_mode">list,form</field>
</record>
```

¿Qué es una acción?

Es una **ventana que se abre** cuando el usuario entra al módulo. En este caso:

Campo	Significado
model	Siempre ir.actions.act_window para acciones de ventana.
id	Identificador único para usarlo desde los menús.
name	Nombre que aparecerá en la pestaña del navegador.
res_model	Modelo a mostrar: lista_tareas.lista.
view_mode	Vistas a mostrar: lista y formulario.

2. Menús del módulo

Aquí definimos cómo navegar por el módulo en la interfaz de Odoo.

```
<menuitem id="lista_tareas_menu_root" name="Listado de tareas" sequence="10"/>
<menuitem id="lista_tareas_menu_1" name="Gestión de tareas"
parent="lista_tareas_menu_root" sequence="20"/>
<menuitem id="lista_tareas_menu_1_list" name="Ver tareas"
parent="lista_tareas_menu_1" action="action_lista_tareas" sequence="30"/>
```

Elemento XML	Explicación
menuitem	Crea una entrada de menú en la interfaz de Odoo.
id	Identificador único del menú.

name	Nombre que se muestra en pantalla.
parent	Si tiene este atributo, el menú se mostrará dentro de otro.
action	Acción que se ejecuta al hacer clic. Aquí abre la vista lista+form del modelo lista_tareas.lista.
sequence	Número que define el orden de aparición. Los más bajos aparecen primero.

En este caso, el usuario verá: **Listado de tareas** → **Gestión de tareas** → **Ver tareas**

3. Vista tipo Lista (list)

Se muestra cuando se entra al módulo. Es como una tabla con todas las tareas.

```
<record id="view_lista_tareas_list" model="ir.ui.view">
  <field name="name">lista.tareas.list</field>
  <field name="model">lista_tareas.lista</field>
  <field name="arch" type="xml">
    <list string="Tareas" default_order="prioridad desc">
      <field name="tarea"/>
      <field name="prioridad"/>
      <field name="urgente"/>
      <field name="realizada"/>
    </list>
  </field>
</record>
```

Explicación:

- **id**: Identificador único de la vista.
- **model**: El modelo al que pertenece esta vista.
- **default_order="prioridad desc"**: Las tareas se ordenan de mayor a menor prioridad automáticamente.
- Dentro de **<list>** definimos las **columnas** que tendrá la tabla.

Cada fila de la lista representa una tarea, y cada campo es una columna.

4. Vista tipo Formulario (form)

Se usa para ver o editar una tarea individual.

```
<record id="view_lista_tareas_form" model="ir.ui.view">
  <field name="name">lista.tareas.form</field>
  <field name="model">lista_tareas.lista</field>
  <field name="arch" type="xml">
    <form string="Tarea">
      <sheet>
```

```

    <group>
      <field name="tarea"/>
      <field name="realizada"/>
    </group>
    <group string="Prioridad y urgencia">
      <field name="prioridad"/>
      <field name="urgente" readonly="1"/>
    </group>
  </sheet>
</form>
</field>
</record>

```

Detalles importantes:

- <sheet>: Contenedor visual principal del formulario.
- <group>: Agrupa campos en bloques, visualmente agradables.
- readonly="1": Hace que el campo no se pueda editar (importante para campos calculados como urgente).



5. Vista de búsqueda (search)

Se usa para buscar o filtrar registros.

```

<record id="view_lista_tareas_search" model="ir.ui.view">
  <field name="name">lista.tareas.search</field>
  <field name="model">lista_tareas.lista</field>
  <field name="arch" type="xml">
    <search string="Buscar tareas">
      <field name="tarea"/>
      <filter name="filtro_urgente" string="Urgente"
domain="(['urgente','=',True)]"/>
      <filter name="filtro_realizada" string="Realizadas"
domain="(['realizada','=',True)]"/>
    </search>
  </field>
</record>

```

Elementos clave:

- <field name="tarea"/>: Permite buscar por nombre de tarea.
- <filter>: Filtros rápidos. En este caso:
 - Urgente: muestra solo tareas urgentes.
 - Realizadas: muestra solo tareas completadas.



Resultado en la interfaz de Odoo

Con todo lo anterior, el usuario podrá:

1. Entrar al menú **Listado de tareas**.
2. Ver una tabla de tareas ordenadas por prioridad.

3. Hacer clic en una tarea para editarla.
4. Ver si es urgente (basado en su prioridad).
5. Buscar tareas por nombre, urgencia o estado.

8. SEGURIDAD EN ODOO: IR.MODEL.ACCESS.CSV

¿Por qué es necesario este archivo?

En Odoo, aunque crees un modelo y vistas, **ningún usuario podrá acceder a ese modelo** si no se definen explícitamente los permisos. Estos permisos se configuran en un archivo **CSV (valores separados por comas)** llamado `ir.model.access.csv`.

Este archivo le indica al sistema **qué grupos de usuarios** pueden:

Permiso	Significado
read	Ver los registros del modelo.
write	Modificar registros existentes.
create	Crear nuevos registros.
unlink	Eliminar registros.

Contenido típico del archivo `ir.model.access.csv`

Aunque no lo podemos mostrar completamente porque el archivo no está accesible directamente, te presento un ejemplo basado en tu módulo:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_lista_tareas,access.lista.tareas,model_lista_tareas_lista,,1,1,1,1
```

Explicación línea por línea:

Campo	Significado
id	ID técnico de esta regla de acceso.
name	Nombre visible (convención: igual que el ID).
model_id:id	Nombre técnico del modelo, precedido por <code>model_</code> . Para tu módulo es <code>model_lista_tareas_lista</code> .

group_id:id	Grupo al que se aplica esta regla. Si se deja vacío (""), aplica a todos los usuarios.
perm_read	1 significa que el permiso de lectura está activado.
perm_write	1 significa que se puede modificar registros.
perm_create	1 permite crear nuevas tareas.
perm_unlink	1 permite borrar tareas.

¿Dónde se enlaza este archivo?

Lo declaraste correctamente en tu `__manifest__.py`:

```
'data': [
    'security/ir.model.access.csv',
    'views/views.xml',
]
```

Esto le dice a Odoo que debe **cargar los permisos** al instalar el módulo.

9. RESUMEN FINAL DEL MÓDULO

Aquí tienes un resumen esquemático de lo que hace este módulo y cómo lo implementamos:

Elemento	Descripción
Nombre del módulo	Lista de Tareas
Finalidad	Crear y gestionar tareas con prioridad, urgencia (calculada) y estado de realización.
Modelo	lista_tareas.lista
Campos	tarea, prioridad, urgente (calculado), realizada
Vista lista	Tabla que muestra todas las tareas, ordenadas por prioridad.
Vista formulario	Permite crear o editar una tarea con campos agrupados.

Vista de búsqueda	Filtros rápidos: buscar por nombre, urgentes, o realizadas.
Menús	Menú jerárquico: "Listado de tareas" → "Gestión de tareas" → "Ver tareas".
Seguridad	Todos los usuarios tienen acceso completo al modelo (read, write, create, unlink).